Channel Partner Invoice Upload

Roswalt My Home — Current Logic & Gap vs. Synergy Integration

Reference document covering the existing Channel Partner invoice submission flow in the Roswalt My Home app, and what's missing to make submitted invoices actually land in the Synergy app.

Key finding: The invoice submission flow is currently UI-only. No backend call is made anywhere in this flow — submitted invoices are held only in an in-memory list for the current app session and are lost on restart. There is no existing connection, past or present, between this flow and Synergy.

1. Where This Lives in the Codebase

File: `lib/screens/partner/cp_payouts_screen.dart`

Two classes in this one file:

a) CPPayoutsScreen — the Payouts list. Shows previously "submitted" invoices (in-memory only, field `_invoices`). A floating + button opens the submission form and inserts the result at the top of the list on return.

b) CPSubmitInvoiceScreen — the submission form itself.

2. Form Fields Collected

Section	Fields
Partner details	Name, Mobile, Firm Name, MahaRERA Registration No., PAN Card No., GST No., Invoice Date
Bank details	Bank Name, Account Holder Name, Account Number, Account Type (Savings/Current), Branch, IFSC Code
Attachment	One PDF file (invoice), picked via file picker, max 10MB

3. File Attachment Logic

```
Future<void> _pickPdf() async {
  final result = await FilePicker.platform.pickFiles(
    type: FileType.custom,
    allowedExtensions: ['pdf'],
  );
  if (result == null || result.files.isEmpty) return;
  final file = result.files.first;
  if ((file.size) > 10 * 1024 * 1024) {
    // show 'File too large. Maximum size is 10MB.'
    return;
  }
  setState(() => _pickedFile = file);
}
```

A separate helper also lets the CP download a reference "proforma invoice" PDF template (generated client-side with the `pdf`/`printing` packages) to fill out and attach alongside their real invoice — this is just

a template generator, not part of the upload itself.

4. Submission Logic — Where the Gap Is

Full current implementation of `_submit()`:

```
Future<void> _submit() async {
  if (_nameCtrl.text.trim().isEmpty || _mobileCtrl.text.trim().isEmpty) {
    // show 'Please fill at least Name and Mobile number.'
    return;
  }
  if (_pickedFile == null) {
    // show 'Please attach your invoice PDF.'
    return;
  }

  setState(() => _submitting = true);

  // NOTE: No backend endpoint exists yet for invoice submission.
  // This currently just simulates a brief delay and returns the
  // entered details to the Payouts list locally. Replace this block
  // with a real API call (multipart upload of _pickedFile plus the
  // form fields) once the endpoint is available.
  await Future.delayed(const Duration(milliseconds: 800));

  setState(() => _submitting = false);
  // show 'Invoice added locally. Backend submission is not yet connected.'

  Navigator.pop(context, {
    'invoiceNo': 'INV_' + DateTime.now().millisecondsSinceEpoch.toString().substring(7),
    'date': _fmtDate(_date),
  });
}
```

The returned map is inserted into `CPPayoutsScreen._invoices` with status `SUBMITTED` — purely local UI state, not persisted to Firestore, Salesforce, or any external system.

5. "Synergy" in This Codebase — Unrelated to Invoices Today

A repo-wide search for "synergy" turns up only branding assets (logo/intro video) and the **employee ticketing module** — a completely separate system from CP invoices/payouts. It has its own backend, its own auth, and no code path touches invoices.

File: `lib/services/ticket_api_service.dart`

```
static const _base = 'https://sales-api.roswaltsmartcue.com';
// own email-OTP session, own bearer token (flutter_secure_storage),
// own host - entirely separate from Firebase / My Home auth.
```

6. Closest Working Multipart-Upload Pattern to Reuse

The ticketing module does have a real, working multipart file-upload call. This is the pattern to model a real Synergy invoice upload on, once an actual Synergy ingestion endpoint exists:

```
Future<Ticket> addAttachmentFile(String id, File file) async {
  final uri = Uri.parse('\$_base/api/v1/tickets/\$id/attachments/file');
  final req = http.MultipartRequest('POST', uri);
```

```
    if (_token != null) req.headers['Authorization'] = 'Bearer \$_token';
    req.files.add(await http.MultipartFile.fromPath('file', file.path));
    final streamed = await req.send().timeout(const Duration(seconds: 60));
    final res = await http.Response.fromStream(streamed);
    return Ticket.fromJson(_decode(res) as Map<String, dynamic>);
  }
```

7. What's Needed to Actually Wire This to Synergy

- The Synergy invoice-ingestion endpoint URL (e.g. POST /api/v1/invoices or similar)
- Its auth scheme (bearer token? API key? shared secret?) and how this app would obtain/store it
- Expected multipart field names: the file field key, plus which form fields (name, mobile, firm, RERA, PAN, GST, bank details, invoice date) it accepts and in what shape (flat fields vs. nested JSON payload alongside the file)
- Expected response shape (success confirmation, invoice ID/number assigned by Synergy, error format)
- Whether Synergy expects the proforma template data merged into the request, or only the raw attached PDF

Once that spec is available (ideally documented the way the ticketing module's Roswalt_Ticketing_Integration_Spec.pdf documents its own endpoints), `CPSubmitInvoiceScreen._submit()` can be rewritten to perform a real multipart POST following the pattern in section 6, replacing the current simulated delay.